

UNIVERZA NA PRIMORSKEM  
FAKULTETA ZA MATEMATIKO, NARAVOSLOVJE IN  
INFORMACIJSKE TEHNOLOGIJE

Project Report

# Huffman block coding (song)

Huffman-Based Lossless Compression of WAV Files

**AUTHORS:**

Ema Čikotić 89221136

Nade Belovinova 89231099

Samoil Gjorgiev 89231015

**PROFESSOR:**

dr. Enes Pasalić

**ASSISTANT:**

Dilawar Abbas Khan

Koper, May 2026

# Contents

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                         | <b>2</b>  |
| <b>2</b>  | <b>Problem Description</b>                  | <b>2</b>  |
| <b>3</b>  | <b>Project Approach</b>                     | <b>2</b>  |
| <b>4</b>  | <b>Technologies Used</b>                    | <b>2</b>  |
| <b>5</b>  | <b>Project Structure</b>                    | <b>3</b>  |
| <b>6</b>  | <b>Code Explanation</b>                     | <b>3</b>  |
| 6.1       | main.c . . . . .                            | 3         |
| 6.2       | buffer.c and buffer.h . . . . .             | 5         |
| 6.3       | searchdata.c and searchdata.h . . . . .     | 6         |
| 6.4       | frequency.c and frequency.h . . . . .       | 7         |
| 6.5       | huffman_tree.c and huffman_tree.h . . . . . | 8         |
| 6.6       | code_table.c and code_table.h . . . . .     | 10        |
| 6.7       | encoder.c and encoder.h . . . . .           | 12        |
| <b>7</b>  | <b>Custom Huffman File Format</b>           | <b>14</b> |
| <b>8</b>  | <b>Results</b>                              | <b>15</b> |
| <b>9</b>  | <b>Challenges Encountered</b>               | <b>15</b> |
| <b>10</b> | <b>Conclusion</b>                           | <b>15</b> |

# 1 Introduction

This project focuses on implementing Huffman-based lossless compression for WAV audio files using the C programming language.

The application reads raw audio data from WAV files, performs frequency analysis on the audio bytes, constructs a Huffman tree, generates Huffman codes, and finally encodes the data into a compressed `.huff` file.

The project was developed as part of the TOR III (Theory of Information) course and combines several important concepts from information theory and low-level programming. During the implementation, concepts such as lossless compression, binary tree structures, MinHeap data structures, frequency analysis, file parsing, dynamic memory allocation, and bit-level encoding were explored and applied in practice.

## 2 Problem Description

The objective of this project was to implement a Huffman-based lossless compression system for WAV audio files.

Audio files stored in uncompressed formats such as WAV can require a large amount of storage space. Because of this, efficient compression methods are important for reducing file size while still preserving the original information.

The project focuses on Huffman coding, a lossless compression technique that assigns shorter binary codes to more frequently occurring symbols and longer codes to less common symbols.

Since WAV files contain raw uncompressed audio data, they are suitable for frequency analysis and Huffman-based encoding.

## 3 Project Approach

The implementation of the project follows the standard Huffman compression pipeline.

The program first reads the WAV file into memory and dynamically locates the "data" chunk containing the raw audio samples. Once the audio data is extracted, the application iterates through all bytes and generates a frequency table that records how often each byte value appears.

Using the generated frequency table, a Huffman tree is constructed with the help of a MinHeap data structure. The tree is then traversed recursively in order to generate Huffman codes for every symbol present in the audio data.

Finally, the generated codes are used to encode the original audio into a compressed bitstream, which is then written into a custom `.huff` output file together with additional metadata required for decoding.

## 4 Technologies Used

The project was implemented entirely in the C programming language using the GCC compiler.

Several low-level programming concepts and data structures were used throughout the implementation, including binary trees, MinHeap structures, dynamic memory allocation, recursive tree traversal, and bit-level file encoding.

The technologies and main components used in the project are summarized below.

| Component            | Technology            |
|----------------------|-----------------------|
| Programming Language | C                     |
| Compiler             | GCC                   |
| Data Structures      | Binary Trees, MinHeap |
| File Format          | WAV                   |
| Compression Method   | Huffman Coding        |

## 5 Project Structure

The project is organized into multiple modules in order to improve readability, maintainability, and separation of functionality.

Each module is responsible for a specific part of the compression pipeline, including file reading, frequency analysis, Huffman tree construction, code generation, and bit-level encoding.

The overall structure of the project is shown below:

```
huffman-block-coding-audio/  
|-- main.c  
|-- buffer.c  
|-- buffer.h  
|-- frequency.c  
|-- frequency.h  
|-- huffman_tree.c  
|-- huffman_tree.h  
|-- code_table.c  
|-- code_table.h  
|-- encoder.c  
|-- encoder.h  
|-- searchdata.c  
|-- searchdata.h  
`-- README.md
```

## 6 Code Explanation

GitHub repository: <https://github.com/EmaCikotic/huffman-block-coding-audio>

### 6.1 main.c

The `main.c` file contains the main execution pipeline of the application and coordinates all stages of the Huffman compression process.

The program begins by reading the WAV audio file into memory using the file buffer module. After loading the file, the application searches for the "data" chunk containing the raw audio samples.

Once the audio data is extracted, the program generates a frequency table describing how many times each byte value appears in the audio stream. These frequencies are then used to construct the Huffman tree.

After the tree is created, the application generates Huffman codes for every symbol and stores them in a code table. Finally, the audio data is encoded using the generated codes and written into a compressed `.huff` file.

The file also provides console output describing each stage of execution in order to simplify debugging and demonstrate the compression pipeline.

```
#include <stdio.h>
#include <stdlib.h>
#include "buffer.h"
#include "searchdata.h"
#include "frequency.h"
#include "huffman_tree.h"
#include "code_table.h"
#include "encoder.h"

#define WAV_FILE "Kanye West - Cant Tell Me Nothing.wav"
#define OUT_FILE "output.huff"

// gcc -Wall -o huffman main.c buffer.c searchdata.c frequency.c huffman_tree.c
// code_table.c

int main(void) {

    printf("[ Step 1 ] Reading WAV file: %s\n", WAV_FILE);
    FileBuffer wav = readFile(WAV_FILE);
    if (!wav.data) {
        fprintf(stderr, "Failed to read WAV file. Exiting.\n");
        return 1;
    }

    DataChunk dc = findDataChunk(wav.data, wav.size);
    if (dc.offset < 0) {
        fprintf(stderr, "Error: 'data' chunk not found.\n");
        freeFileBuffer(&wav);
        return 1;
    }

    unsigned char *audioData = wav.data + dc.offset;
    long audioSize = (long)dc.size;

    FileBuffer audioBuffer;
    audioBuffer.data = audioData;
    audioBuffer.size = audioSize;

    printf(" WAV file size : %ld bytes\n", wav.size);
    printf(" Audio data at : byte %ld\n", dc.offset);
    printf(" Audio size      : %u bytes\n", dc.size);

    printf("\n[ Step 2 ] Counting the byte frequencies...\n");

    FreqTable freq;
    buildFrequencyTable(audioData, audioSize, freq);
    printf("Frequency table built successfully. \n");

    printf("\n[ Step 3 ] Building Huffman tree...\n");

    HuffNode *root = buildHuffmanTree(freq);

    if (!root) {
        fprintf(stderr, "Failed to build Huffman tree. Exiting.\n");
        freeFileBuffer(&wav);
        return 1;
    }

    printf(" Huffman tree built successfully.\n");

    printf("\n[ Step 4 ] Generating Huffman codes...\n");

    CodeTable table;
    buildCodeTable(root, table);

    printf(" Huffman code table generated successfully.\n");
```

```

printf("\nExample Huffman codes:\n");

for (int i = 0; i < 10; i++) {
if (table[i].length > 0) {
    printf("Byte %3d : %s (%d bits)\n",
        i,
        table[i].code,
        table[i].length);
}

}

printf("[ Step 5 ] Encoding and writing '%s'...\n", OUT_FILE);

long huffSize = encodeToFile(&audioBuffer, table, OUT_FILE);
if (huffSize == 0) {
    fprintf(stderr, "Encoding failed. Exiting.\n");
    freeHuffmanTree(root);
    freeFileBuffer(&wav);
    return 1;
}
printf("    Output file size: %ld bytes (%.2f MB)\n\n", huffSize,
    (double)huffSize / (1024.0 * 1024.0));

freeHuffmanTree(root);
freeFileBuffer(&wav);
return 0;
}

```

## 6.2 buffer.c and buffer.h

The `buffer` module is responsible for reading files into memory and managing dynamically allocated buffers.

The `readFile()` function opens the input WAV file in binary mode, determines its size, allocates enough memory for the entire file, and loads all bytes into memory. This allows the rest of the application to directly access raw audio data during processing.

The `freeFileBuffer()` function safely releases allocated memory after processing is complete in order to avoid memory leaks.

The `FileBuffer` structure stores both the raw byte data and the size of the loaded file.

The `freeFileBuffer()` function safely releases allocated memory after processing is complete.

```

#include <stdio.h>
#include <stdlib.h>
#include "buffer.h"

//https://www.freeconvert.com/mp3-to-wav goats.

FileBuffer readFile(const char *filename) {
    FileBuffer fb = { NULL, 0 };

    FILE *file = fopen(filename, "rb");
    if (!file) {
        printf("Error: Cannot open file '%s'\n", filename);
        return fb;
    }
}

```

```

// --- figure out how big the file is ---
fseek(file, 0, SEEK_END);
fb.size = ftell(file);
rewind(file);

// --- allocate exactly that many bytes ---
fb.data = (unsigned char *)malloc(fb.size);
if (!fb.data) {
    printf("Error: Memory allocation failed\n");
    fclose(file);
    fb.size = 0;
    return fb;
}

// --- read the whole thing in one shot ---
size_t bytesRead = fread(fb.data, 1, fb.size, file);
if ((long)bytesRead != fb.size) {
    printf("Warning: Expected %ld bytes but only read %zu\n", fb.size,
        bytesRead);
}

fclose(file);
return fb;
}

void freeFileBuffer(FileBuffer *fb) {
    if (fb->data) {
        free(fb->data);
        fb->data = NULL;
    }
    fb->size = 0;
}

```

```

#ifndef BUFFER_H
#define BUFFER_H

typedef struct {
    unsigned char *data;    // raw bytes
    long          size;     // how many bytes
} FileBuffer;

FileBuffer readFile      (const char *filename);
void          freeFileBuffer (FileBuffer *fb);

#endif

```

## 6.3 searchdata.c and searchdata.h

The `searchdata` module is responsible for locating the "data" chunk inside the WAV file.

Since WAV files may contain additional metadata sections before the actual audio samples, relying on a fixed byte offset is not always reliable. The implemented solution dynamically searches through the file and detects the beginning of the audio data.

The `findDataChunk()` function scans the raw file bytes until it finds the "data" identifier. After locating the chunk, the function extracts the offset and size of the audio data section.

The `DataChunk` structure stores the position and size of the discovered audio block.

```

#include <stdio.h>
#include "searchdata.h"

DataChunk findDataChunk(unsigned char *data, long size) {
    DataChunk result;
    result.offset = -1;
    result.size = 0;

    for (long i = 0; i < size - 8; i++) {
        if (data[i] == 'd' &&
            data[i + 1] == 'a' &&
            data[i + 2] == 't' &&
            data[i + 3] == 'a') {

            unsigned int chunkSize =
                data[i + 4] |
                (data[i + 5] << 8) |
                (data[i + 6] << 16) |
                (data[i + 7] << 24);

            printf("Found 'data' chunk at byte %ld (size = %u bytes)\n",
                i, chunkSize);

            result.offset = i + 8;
            result.size = chunkSize;

            return result;
        }
    }

    return result;
}

```

```

#ifndef SEARCHDATA_H
#define SEARCHDATA_H

typedef struct {
    long offset;
    unsigned int size;
} DataChunk;

DataChunk findDataChunk(unsigned char *data, long size);

#endif

```

## 6.4 frequency.c and frequency.h

The `frequency` module is responsible for counting how many times each byte value appears in the audio data.

The implementation uses a frequency table containing 256 entries, where each position corresponds to one possible byte value between 0 and 255.

The `buildFrequencyTable()` function iterates through all audio bytes and increments the corresponding counter for every encountered symbol.

The generated frequency table is later used for constructing the Huffman tree and generating variable-length Huffman codes.

```

#include <stdio.h>
#include <string.h>
#include "frequency.h"

void buildFrequencyTable(const unsigned char *data, long size, FreqTable freq) {
    memset(freq, 0, sizeof(FreqTable));
}

```

```

    for (long i = 0; i < size; i++) {
        freq[ data[i] ]++;
    }
}

```

```

#ifndef FREQUENCY_H
#define FREQUENCY_H
#define NUM_SYMBOLS 256

typedef unsigned long long FreqTable[NUM_SYMBOLS];

void    buildFrequencyTable (const unsigned char *data, long size, FreqTable freq);

#endif

```

## 6.5 huffman\_tree.c and huffman\_tree.h

The `huffman_tree` module implements the Huffman tree data structure together with the MinHeap used during tree construction.

Each node of the Huffman tree is represented using the `HuffNode` structure, which stores:

- the symbol value
- the symbol frequency
- pointers to the left and right child nodes

The implementation uses a MinHeap priority queue in order to efficiently construct the Huffman tree.

Initially, one leaf node is created for every symbol that appears in the audio data. The two nodes with the smallest frequencies are repeatedly extracted from the heap and merged into a new parent node whose frequency equals the sum of both child frequencies.

This process continues until only one node remains, which becomes the root of the Huffman tree.

The module also implements recursive memory deallocation through the `freeHuffmanTree()` function in order to safely release all dynamically allocated nodes after processing is complete.

```

#include <stdio.h>
#include <stdlib.h>
#include "huffman_tree.h"

typedef struct {
    HuffNode *nodes[NUM_SYMBOLS * 2];
    int      size;
} MinHeap;

static HuffNode *newNode(int symbol, unsigned long long freq) {
    HuffNode *n = (HuffNode *)malloc(sizeof(HuffNode));
    if (!n) { fprintf(stderr, "Out of memory\n"); exit(1); }
    n->symbol = symbol;
    n->freq   = freq;
    n->left   = NULL;
    n->right  = NULL;
    return n;
}

```

```

static void swap(MinHeap *h, int i, int j) {
    HuffNode *tmp = h->nodes[i];
    h->nodes[i] = h->nodes[j];
    h->nodes[j] = tmp;
}

static void bubbleUp(MinHeap *h, int idx) {
    while (idx > 0) {
        int parent = (idx - 1) / 2;
        if (h->nodes[parent]->freq > h->nodes[idx]->freq) {
            swap(h, parent, idx);
            idx = parent;
        } else {
            break;
        }
    }
}

static void sinkDown(MinHeap *h, int idx) {
    int size = h->size;
    while (1) {
        int left = 2 * idx + 1;
        int right = 2 * idx + 2;
        int smallest = idx;

        if (left < size && h->nodes[left]->freq < h->nodes[smallest]->freq)
            smallest = left;
        if (right < size && h->nodes[right]->freq < h->nodes[smallest]->freq)
            smallest = right;

        if (smallest != idx) {
            swap(h, smallest, idx);
            idx = smallest;
        } else {
            break;
        }
    }
}

static void heapInsert(MinHeap *h, HuffNode *node) {
    h->nodes[h->size] = node;
    bubbleUp(h, h->size);
    h->size++;
}

static HuffNode *heapExtractMin(MinHeap *h) {
    HuffNode *min = h->nodes[0];
    h->size--;
    h->nodes[0] = h->nodes[h->size];
    sinkDown(h, 0);
    return min;
}

HuffNode *buildHuffmanTree(const FreqTable freq) {
    MinHeap heap;
    heap.size = 0;

    int leafCount = 0;
    for (int b = 0; b < NUM_SYMBOLS; b++) {
        if (freq[b] > 0) {
            heapInsert(&heap, newNode(b, freq[b]));
            leafCount++;
        }
    }

    printf("Building Huffman tree from %d distinct symbols...\n", leafCount);

    if (heap.size == 1) {
        HuffNode *only = heapExtractMin(&heap);
        HuffNode *root = newNode(-1, only->freq);
        root->left = only;
        return root;
    }
}

```

```

while (heap.size > 1) {
    HuffNode *left = heapExtractMin(&heap);
    HuffNode *right = heapExtractMin(&heap);

    HuffNode *parent = newNode(-1, left->freq + right->freq);
    parent->left = left;
    parent->right = right;

    heapInsert(&heap, parent);
}

return heapExtractMin(&heap);
}

void freeHuffmanTree(HuffNode *root) {
    if (!root) return;
    freeHuffmanTree(root->left);
    freeHuffmanTree(root->right);
    free(root);
}

```

```

#ifndef HUFFMAN_TREE_H
#define HUFFMAN_TREE_H

#include "frequency.h"

typedef struct HuffNode {
    int symbol; // byte value 0-255, or -1 for internal nodes
    unsigned long long freq; // frequency of this subtree
    struct HuffNode *left; // 0-branch
    struct HuffNode *right; // 1-branch
} HuffNode;

HuffNode *buildHuffmanTree (const FreqTable freq);
void freeHuffmanTree (HuffNode *root);

#endif

```

## 6.6 code\_table.c and code\_table.h

The `code_table` module is responsible for generating Huffman codes from the constructed Huffman tree.

The implementation recursively traverses the tree starting from the root node. During traversal:

- moving to the left child appends a binary 0
- moving to the right child appends a binary 1

When a leaf node is reached, the accumulated binary path represents the final Huffman code for that symbol.

The generated codes are stored inside the `CodeTable` structure together with their corresponding code lengths.

These Huffman codes are later used during the encoding stage to compress the audio data into a variable-length bitstream.

```

#include <stdio.h>
#include <string.h>
#include "code_table.h"

static void walk(HuffNode *root, CodeTable table, char *path, int depth) {
    if (!root) return;

    if (root->symbol != -1) {
        // --- LEAF NODE: save the path we accumulated as this symbol's code ---
        path[depth] = '\0'; // null-terminate the string
        strncpy(table[root->symbol].code, path, MAX_CODE_LEN - 1);
        table[root->symbol].length = depth;
        return;
    }

    // --- INTERNAL NODE: go left (add '0'), then right (add '1') ---
    path[depth] = '0';
    walk(root->left, table, path, depth + 1);

    path[depth] = '1';
    walk(root->right, table, path, depth + 1);
}

void buildCodeTable(HuffNode *root, CodeTable table) {
    // initialise every entry to empty
    for (int b = 0; b < NUM_SYMBOLS; b++) {
        table[b].code[0] = '\0';
        table[b].length = 0;
    }

    char path[MAX_CODE_LEN];
    walk(root, table, path, 0);
}

/*void printCodeTable(const CodeTable table) {
    printf("\n--- Huffman Code Table ---\n");
    printf("%-10s %-12s %-10s %s\n", "Byte(dec)", "Byte(hex)", "CodeLen", "Code");
    printf("-----\n");

    for (int b = 0; b < NUM_SYMBOLS; b++) {
        if (table[b].length > 0) {
            printf("%-10d 0x%-10X %-10d %s\n",
                b, b, table[b].length, table[b].code);
        }
    }
    printf("-----\n\n");
}
*/

```

```

#ifndef CODE_TABLE_H
#define CODE_TABLE_H

#include "huffman_tree.h"

#define MAX_CODE_LEN 256 // safety ceiling for code strings

typedef struct {
    char code[MAX_CODE_LEN]; // e.g. "01101"
    int length; // number of bits in the code
} CodeEntry;

// codeTable[b] = the Huffman code for byte value b
typedef CodeEntry CodeTable[NUM_SYMBOLS];

void buildCodeTable (HuffNode *root, CodeTable table);
//void printCodeTable (const CodeTable table);

#endif

```

## 6.7 encoder.c and encoder.h

The `encoder` module is responsible for converting the original audio data into a compressed Huffman-encoded bitstream and writing the result into a custom `.huff` file.

The implementation uses a custom `BitWriter` structure which allows individual bits to be packed into bytes before writing them to the output file.

For every byte in the original audio data, the corresponding Huffman code is retrieved from the generated code table and written bit-by-bit into the output stream.

The module also stores additional metadata inside the `.huff` file, including:

- the file signature
- symbol information
- Huffman code lengths
- the original file size
- padding information

Since Huffman codes have variable lengths, the final byte of the encoded stream may not be completely filled. The implementation therefore calculates and stores the number of padding bits added to the last byte.

This module represents the final stage of the compression pipeline and produces the compressed output file.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "encoder.h"

typedef struct {
    FILE *file;
    unsigned char currentByte;
    int bitsFilled;
} BitWriter;

static void bwInit(BitWriter *bw, FILE *file) {
    bw->file = file;
    bw->currentByte = 0;
    bw->bitsFilled = 0;
}

static void bwWriteBit(BitWriter *bw, int bit) {
    bw->currentByte = (bw->currentByte << 1) | (bit & 1);
    bw->bitsFilled++;

    if (bw->bitsFilled == 8) {
        fwrite(&bw->currentByte, 1, 1, bw->file);
        bw->currentByte = 0;
        bw->bitsFilled = 0;
    }
}

static int bwFlush(BitWriter *bw) {
    if (bw->bitsFilled == 0) return 0;

    int padding = 8 - bw->bitsFilled;
    bw->currentByte = bw->currentByte << padding;
    fwrite(&bw->currentByte, 1, 1, bw->file);
    bw->bitsFilled = 0;
    return padding;
}
```

```

static unsigned char packCode(const char *code, unsigned char length) {
    unsigned char packed = 0;
    for (int j = 0; j < length; j++) {
        packed = (packed << 1) | (code[j] == '1' ? 1 : 0);
    }
    packed <= (8 - length); // left-align in the byte
    return packed;
}

long encodeToFile(const FileBuffer *fb, const CodeTable table,
                  const char *outputPath) {

    FILE *out = fopen(outputPath, "wb");
    if (!out) {
        printf("Error: Cannot create output file '%s'\n", outputPath);
        return 0;
    }

    fwrite("HUFF", 1, 4, out);

    unsigned char symbolCount = 0;
    for (int b = 0; b < NUM_SYMBOLS; b++) {
        if (table[b].length > 0) symbolCount++;
    }
    fwrite(&symbolCount, 1, 1, out);
    for (int b = 0; b < NUM_SYMBOLS; b++) {
        if (table[b].length > 0) {
            unsigned char sym = (unsigned char)b;
            unsigned char len = table[b].length;
            unsigned char packed = packCode(table[b].code, len);

            fwrite(&sym, 1, 1, out); // 1 byte: symbol value
            fwrite(&len, 1, 1, out); // 1 byte: code length in bits
            fwrite(&packed, 1, 1, out); // 1 byte: the code bits, left-aligned
        }
    }

    fwrite(&fb->size, sizeof(long), 1, out);

    long paddingPos = ftell(out);
    unsigned char paddingByte = 0;
    fwrite(&paddingByte, 1, 1, out);

    BitWriter bw;
    bwInit(&bw, out);

    printf("Encoding %ld bytes...\n", fb->size);

    for (long i = 0; i < fb->size; i++) {
        unsigned char sym = fb->data[i];
        const char *code = table[sym].code;
        int len = table[sym].length;

        for (int j = 0; j < len; j++) {
            bwWriteBit(&bw, code[j] == '1' ? 1 : 0);
        }
    }

    int padding = bwFlush(&bw);

    fseek(out, paddingPos, SEEK_SET);
    paddingByte = (unsigned char)padding;
    fwrite(&paddingByte, 1, 1, out);

    fseek(out, 0, SEEK_END);
    long outputSize = ftell(out);

    fclose(out);
}

```

```

    printf("Encoding done.\n");

    return outputSize;
}

```

```

#ifndef ENCODER_H
#define ENCODER_H

// =====
//  encoder.h      encode the WAV bytes and write a .huff file
// =====
//
//  FILE FORMAT of the .huff output
//  -----
//  [ 4 bytes ] magic number  "HUFF"
//  [ 4 bytes ] number of distinct symbols (int)
//  [ for each symbol ]
//      [ 1 byte ] symbol value (0-255)
//      [ 4 bytes ] code length in bits (int)
//      [ N bytes ] the code string, null-terminated
//  [ 8 bytes ] original file size in bytes (long long)
//  [ 1 byte ] padding bits added to last byte (0-7)
//  [ rest    ] the encoded bitstream, packed into bytes

#include "buffer.h"
#include "code_table.h"

long encodeToFile(const FileBuffer *fb, const CodeTable table, const char
    *outputPath);

#endif

```

## 7 Custom Huffman File Format

The project creates a custom `.huff` file format used to store compressed audio data together with all metadata required for future decoding.

|                            |
|----------------------------|
| <b>magic number</b>        |
| <b>symbol metadata</b>     |
| <b>original file size</b>  |
| <b>padding information</b> |
| <b>encoded bitstream</b>   |

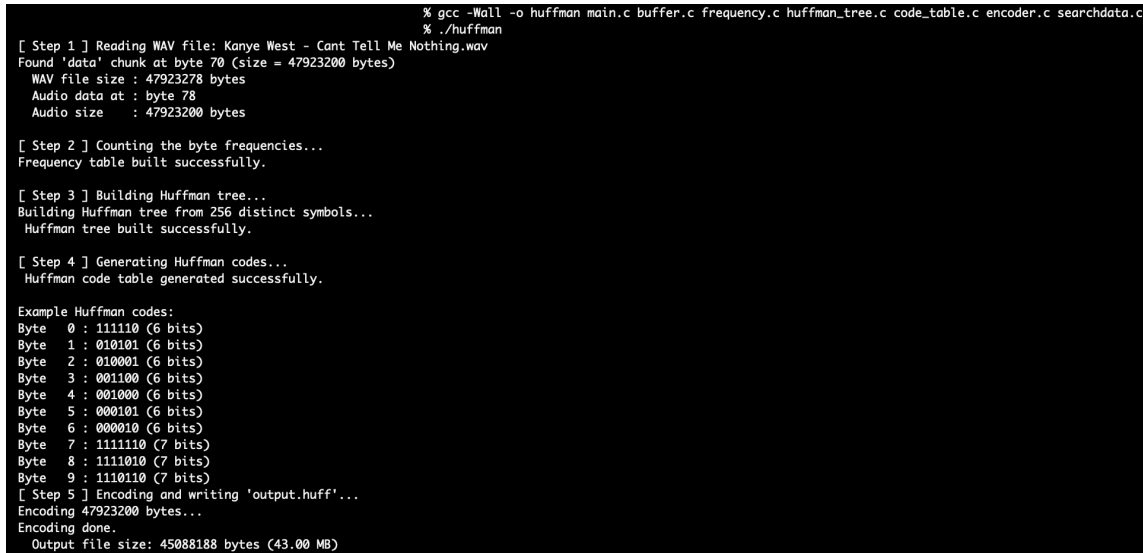
The **magic number** identifies the file as a Huffman-compressed file. The **symbol metadata** stores information about generated Huffman codes and their lengths.

The **original file size** is required during decompression in order to reconstruct the original audio data correctly. The **padding information** stores the number of additional bits added to complete the final byte of the encoded stream.

Finally, the **encoded bitstream** contains the compressed audio data represented using Huffman codes.

## 8 Results

The following screenshot shows the successful compilation and execution of the implemented Huffman compression system, including WAV file processing, Huffman code generation, and creation of the compressed `.huff` output file.



```
% gcc -Wall -o huffman main.c buffer.c frequency.c huffman_tree.c code_table.c encoder.c searchdata.c
% ./huffman
[ Step 1 ] Reading WAV file: Kanye West - Cant Tell Me Nothing.wav
Found 'data' chunk at byte 70 (size = 47923200 bytes)
WAV file size : 47923278 bytes
Audio data at : byte 78
Audio size : 47923200 bytes

[ Step 2 ] Counting the byte frequencies...
Frequency table built successfully.

[ Step 3 ] Building Huffman tree...
Building Huffman tree from 256 distinct symbols...
Huffman tree built successfully.

[ Step 4 ] Generating Huffman codes...
Huffman code table generated successfully.

Example Huffman codes:
Byte 0 : 111110 (6 bits)
Byte 1 : 010101 (6 bits)
Byte 2 : 010001 (6 bits)
Byte 3 : 001100 (6 bits)
Byte 4 : 001000 (6 bits)
Byte 5 : 000101 (6 bits)
Byte 6 : 000010 (6 bits)
Byte 7 : 1111110 (7 bits)
Byte 8 : 1111010 (7 bits)
Byte 9 : 1110110 (7 bits)
[ Step 5 ] Encoding and writing 'output.huff'...
Encoding 47923200 bytes...
Encoding done.
Output file size: 45088188 bytes (43.00 MB)
```

Figure 1: Successful execution of the Huffman audio compression pipeline

The original WAV file size was 47,923,278 bytes, while the compressed `.huff` file size was 45,088,188 bytes. This represents a compression ratio reduction of approximately 5.9%.

## 9 Challenges Encountered

Several challenges appeared during development of the project.

One difficulty was understanding the internal structure of WAV files and correctly locating the "data" chunk containing the raw audio samples.

Another challenge was implementing the MinHeap used for Huffman tree construction and ensuring that nodes were inserted and removed correctly.

Bit-level encoding also required careful handling, especially when writing variable-length Huffman codes and managing padding bits in the final byte.

The project additionally involved dynamic memory allocation and recursive tree traversal, which required careful debugging and testing.

## 10 Conclusion

This project provided practical experience with Huffman coding and lossless compression techniques through the implementation of a complete compression pipeline in the C programming language.

During development, several important concepts from information theory and low-level programming were explored, including binary trees, MinHeap data structures, fre-

quency analysis, recursive traversal algorithms, dynamic memory allocation, file parsing, and bit-level operations.

By implementing the compression system manually, the project demonstrated how theoretical concepts discussed in the TOR III course can be applied in practice to build a functioning software solution.

The final implementation successfully reads WAV audio files, constructs Huffman codes based on byte frequencies, and generates compressed `.huff` output files while preserving the original information.

Overall, the project provided practical insight into how real-world compression systems operate internally and how efficient data representation can reduce storage requirements while preserving the original information.